

协同驾驭：大语言模型智能体的驾驭器与模型权重协同演化

Zhengyu Chen^{1,†}, Teng Xiao^{2,†}, Huaisheng Zhu¹, Yige Yuan³, Luan Zhang¹, Jingang Wang¹

¹Meituan, ²Allen Institute for AI, ³Independent

[†]Equal contribution.

摘要。面向自动化人工智能研究的智能体后训练不仅需要优化模型参数，还需优化运行时驾驭器，该驾驭器决定了研究轨迹的生成、评估与学习方式。现有流程通常在固定驾驭器下训练模型，包括提示、工具、技能、中间件和记忆，而将数据生成过程排除在优化目标之外。这导致模型更新与决定轨迹质量的静态脚手架之间产生不匹配。本文提出协同驾驭框架，在后训练阶段联合优化智能体驾驭器与模型参数。协同驾驭在驾驭器优化与模型优化之间交替进行。基于大语言模型的驾驭器批评器分析失败轨迹，识别驾驭器层面的失效模式，并提出经过验证的局部更新。随后，模型在改进后的驾驭器生成的高质量轨迹上进行微调，将有效的脚手架蒸馏到模型参数中。一项超过 200 小时的自主案例研究进一步表明，协同驾驭能够从系统崩溃中恢复、提升推理效率，并在无需人工干预的情况下发现集成策略。这些结果表明，联合优化驾驭器与模型是超越固定驾驭器后训练的有效途径。

1 引言

智能体的后训练不仅涉及模型权重的训练，还涉及围绕模型的运行时系统设计。在实践中，智能体由两个耦合组件构成：模型参数与用于引导、执行、验证并记录其行为的框架（Harness）。该框架包括提示、工具、可复用技能、中间件、重试逻辑、上下文管理以及记忆。当前的后训练流程对这些组件的处理并不对称。模型通过监督微调、强化学习或偏好优化进行更新，而框架通常由人工设计并保持不变。这造成了不匹配：框架决定了用于训练的轨迹，却未从训练中观察到的失败中改进。糟糕的工具模式、缺失的重试钩子、过短的轮次限制或不够明确的提示，都可能阻碍有用轨迹的生成。随着模型在软件工程和工具使用场景中能力日益增强，固定的框架成为自动化人工智能研发的重要瓶颈 [1]。

Co-evolve Agent Harness and Weights

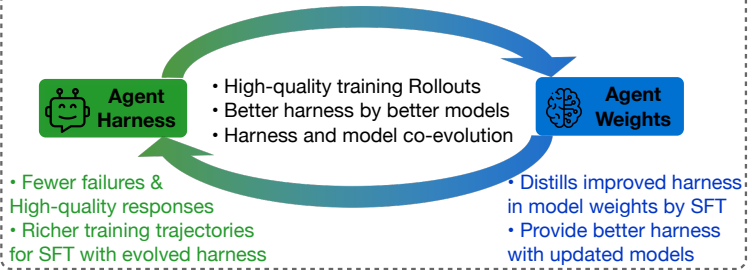


图 1：协同驾驭双环概览。

近期研究表明，通过编辑模型周围的系统而非模型本身，可以提升智能体性能。一类工作为此类编辑暴露了狭窄的接口：利用执行反馈修订提示、演示或语言模型程序 [2 – 5]，而上下文智能体系统将过往经验存储为文本工件，在测试时复用 [6, 7]。较新的工作将这一可编辑接口从单一提示类组件扩展至完整的智能体框架。元框架（Meta-Harness）[8] 从

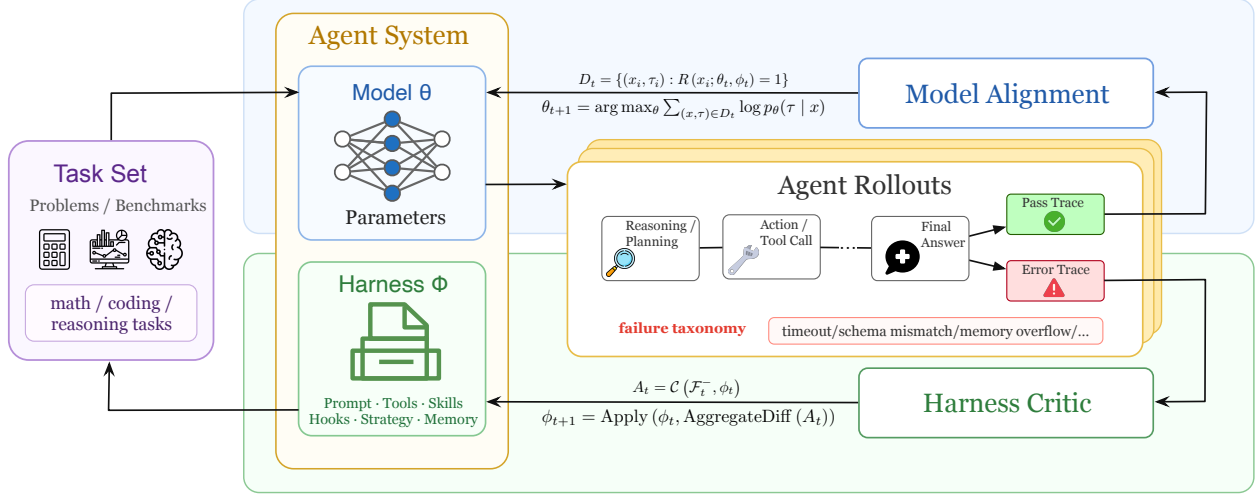


图 2: 协同框架 (Co-Harness) 的总体框架。

源代码、评分和迹中优化框架代码。智能体框架工程 [9] 对提示、工具、中间件、技能和记忆执行可观测性驱动的更新。AEVO[10] 将智能体进化视为交互过程，并编辑驱动未来搜索的过程或上下文。这些工作共同清晰地表明，框架不仅是基础设施，更是一个优化变量。然而，优化后的框架仍然包裹着固定模型。它改善了当前模型在推理时的运行方式，但并未纳入产生轨迹、筛选轨迹并更新下一模型的后训练循环。这引出了我们的问题：框架进化能否与模型后训练耦合，从而使生成训练数据的系统与基于该数据训练的模型共同改进？

我们提出协同训练框架 (Co-Harness)，一种用于智能体后训练的简易双环方法。如图 2 所示，每轮从模型 θ_t 和训练框架 ϕ_t 开始。智能体首先在任务集上收集轨迹。失败轨迹被传递给基于大语言模型的框架批评器 (HarnessCritic)，该批评器将每次失败归因于结构化的框架层面原因，如提示歧义、工具模式错误、技能缺失或中间件不匹配。随后框架批评器提出局部框架差异，仅当验证轨迹改善了目标失败模式且未使保留行为退化时，该差异才被接受。被接受的框架 ϕ^* 用于收集高质量轨迹，这些轨迹随后用于微调下一个模型 θ 。更新后的模型进入下一轮，可能暴露新的框架层面瓶颈。核心思想是框架不应被视为固定的数据生成环境。更好的框架产生更干净的工具使用、更少的可避免崩溃和更具信息量的轨迹；这些轨迹训练出更强的模型，进而能利用更丰富的框架：更好的框架 $\rightarrow$ 更好的轨迹 $\rightarrow$ 更强的模型 $\rightarrow$ 更好的框架 $\rightarrow \dots$

这使得框架成为后训练系统中可学习的部分，而非模型周围的静态支架。本文贡献如下：

1. 本文将智能体后训练形式化为模型与其框架的联合优化问题。协同训练框架不再将提示、工具、技能、中间件和记忆视为固定基础设施，而是交替进行基于失败改进框架和基于改进框架生成的轨迹训练模型。
2. 本文引入框架批评器，一种面向局部框架修复的失败驱动过程。给定失败轨迹，框架批评器将每次失败归因于结构化的框架层面原因，并提出经过验证的提示、工具、技能和中间件差异。
3. 本文表明这一简易循环在工具集成推理中带来复合增益。在 AIME24、

AIME25 和 HMMT25 上，两轮协同训练框架在 Qwen3-8B 和 Qwen3-32B 上平均准确率提升+20.4 个百分点，其中 Qwen3-32B 在 HMMT25 上最大增益为+27.2 个百分点。在 200 余小时的自主运行中，协同训练框架还修复了系统崩溃、提高了推理效率，并在无人工干预的情况下发现了一种集成策略。

2 相关工作

工具集成推理（Tool Integrated Reasoning, TIR）。工具集成推理（Tool-Integrated Reasoning, TIR）通过外部计算工具——最常用的是 Python 代码解释器——增强大语言模型，使模型能够在多轮交互中将基于文本的思维链与可执行代码交替进行，从而解决问题 [11–13]。近期研究表明，TIR 在数学基准上显著优于纯文本推理，不仅因为其将算术运算外包，更在于其解锁了全新的问题求解策略，如迭代搜索、动态规划（DP）和假设驱动的探索 [14]。从理论上讲，TIR 严格扩展了模型的可行支持集：对于任意有限的令牌预算，存在某些算法策略，其程序化表示简洁，而自然语言模拟则冗长到难以处理 [14]。这一差距使得 Harness——即协调工具调用格式、执行及反馈回推理过程的脚手架——成为 TIR 性能的一阶决定因素。与纯文本智能体不同，在纯文本智能体中 Harness 故障仅降低提示清晰度，而在 TIR 中，单个工具模式错误或重试钩子配置不当就可能中止整个多轮代码解释器会话，因此 Harness 协同演化在此场景下尤为重要。

自动化 Harness 设计。近期工作将搜索空间从单一提示类组件扩展到完整的智能体 Harness。Meta-Harness [8] 利用程序源代码、执行轨迹和任务得分来优化 Harness 实现。Agentic Harness Engineering [9] 使用可观测性信号来修订多个 Harness 组件，包括提示、工具、中间件、技能和记忆。AEVO [10] 将智能体演化框架化为交互式过程，并修改引导未来搜索的流程或上下文。这些方法表明，Harness 设计是在不改变基础模型的情况下改进智能体行为的核心杠杆。相比之下，我们的目标不仅是为固定模型寻找更好的推理时脚手架。Co-Harness 将 Harness 演化纳入后训练过程：被接受的 Harness 编辑会改变为监督而收集的轨迹，而这些轨迹随后被用于更新模型本身。

智能体系统的自动化设计。自动化智能体系统设计（Automated Design of Agentic Systems, ADAS）[15] 提出了元智能体搜索：一个元智能体在代码中迭代编程新的智能体设计，并利用先前设计的存档。用图灵完备代码定义的智能体原则上可以表示任何智能体行为，ADAS 展示了强大的跨领域迁移能力。然而，ADAS 在推理时优化的是上下文任务准确率——它没有考虑所发现的智能体设计是否产生高质量的训练轨迹，也没有进行失败归因。我们的工作互补的：ADAS 搜索更广泛的设计空间，而 HarnessCritic 专注于对已知失败模式进行有针对性的、可解释的修复。一个关键的实用区别在于，HarnessCritic 的搜索信号来自失败轨迹（一种引导式调试），而不仅仅是任务结果分数。

3 方法

Co-Harness 将 Harness 改进和模型改进视为两个交替进行的过程：更好的 Harness 产生更清晰的轨迹，从而训练出更强的模型，而更强的模型又会暴露出在较弱策略下不可见的高阶 Harness 瓶颈。该框架总结于图 2。

3.1 概述与问题形式化

定义 3.1（Harness 配置）。Harness 配置

$\phi \in \Phi$ 是一个结构化的五元组

$$\phi = (P, T, S, \text{Mid}, M),$$

其中 P 表示提示和指令模板，T 表示工具定义和接口模式，S 表示可复用的技能或模块，Mid 表示中间件，M 表示长期记忆策略。这里，中间件捆绑了智能体的运行时控制面：编排器（主循环和协议）、用于是/否拦截和验证的钩子，以及上下文管理逻辑，如压缩、截断或检索路由。我们用 ϕ_t 表示协同进化轮次 t 中的活动 Harness。

令 θ_t 表示轮次 t 的模型参数，令 $R(x; \theta, \phi)$ 表示输入 x 的任务奖励，在我们的实验中根据基准实例化为 pass@1 或解决率。理想目标是联合优化模型和 Harness：
$$(\theta^*, \phi^*) = \operatorname{argmax}_{\theta, \phi \in \Phi} \mathbb{E}_{x \sim \mathcal{X}} [R(x; \theta, \phi)]。$$

直接求解该目标是不切实际的，因为评估 Harness 需要完整的智能体轨迹，奖励是稀疏的，而且 θ 的变化会改变哪些 Harness 编辑是有意义的。因此，我们的关键设计选择是将问题分解为两个交替循环：一个在固定模型的情况下改进 Harness，另一个在固定进化后的 Harness 的情况下改进模型。

3.2 双环协同演化架构

协同利用（Co-Harness）在自动测试框架循环（AutoHarnessLoop）与模型对齐循环（ModelAlignLoop）之间交替进行。在第 t 轮，当前模型 θ_t 与当前测试框架 ϕ_t 交互，生成一批轨迹。失败的轨迹被路由至测试框架批评器（HarnessCritic），由其提出针对测试框架的定向修改建议。经过验证后，被接受的配置记为 ϕ_t^* 。在 ϕ_t^* 下运行 θ_t ，得到轨迹集 $\mathcal{D}_t$ ，该轨迹集成为下一次模型更新的训练信号。更新后的模型 θ_{t+1} 被反馈至下一轮协同利用，并从 $\phi_{t+1} \leftarrow \phi_t^*$ 初始化。

协同利用循环。协同利用循环固定 θ_t 并改进测试框架。其输入是在当前组合 (θ_t, ϕ_t) 下观察到的一组失败。其输出是演化后的测试框架 ϕ_t^* ，以及在该演化配置下收集的刷新后的轨迹集。该循环负责寻找环境层面的修复方案：当失败归因于脚手架而非策略时，修复错误的工具模式、引入可复用技能、调整中间件行为或扩展长期记忆。

模型对齐循环。模型对齐循环固定演化后的测试框架 ϕ_t^* 并改进模型。其输入是在演化后的测试框架下产生的更高质量的轨迹集 $\mathcal{D}_t$ 。其输出是更新后的模型 θ_{t+1} ，该模型内化了改进后的行为分布。第二个循环至关重要：没有它，测试框架优化仍是一个推理时搜索问题；有了它，每次成功的测试框架修订都能转化为持久的模型能力。

交替调度。我们将两个循环重复 T 轮。在实践中，当满足以下任一条件时调度停止：（i）没有候选补丁通过验证，或（ii）连续多轮奖励的边际增益低于一个小阈值。我们的实验使用固定的小轮数，但该公式本身不依赖于特定的时间范围。算法 3 总结了协同演化的一个完整轮次。

3.3 哈内斯批评家：从失败轨迹中演化哈内斯

哈内斯批评家，记作 $\mathcal{C}$ ，将原始失败轨迹转换为结构化证据，用于哈内斯修复。给定在 (θ_t, ϕ_t) 下收集的一批 $\mathcal{F}_t^- = \{\tau_i^-\}$ ，哈内斯批评家处理每条轨迹并返回归因集 $\mathcal{A}_t$ ——一组结构化记录，每个失败对应一条，每条记录指定预测的根本原因、涉及的哈内斯维度、严重性评级、来自轨迹的支持证据以及具体的差异建议。形式上， $\mathcal{A}_t = \mathcal{C}(\mathcal{F}_t^-, \phi_t)$ 。这一阶段使协同哈内斯能够从反复出现的、可解释的错误模式中进行优化，而非无向变异。

Algorithm 3 One round of Co-Harness co-evolution

```
1: Input current model  $\theta_t$  and Harness  $\phi_t$ 
2: Collect failed trajectories  $\mathcal{F}_t^-$  under  $(\theta_t, \phi_t)$ 
3: for  $k = 1, \dots, K$  do
4:    $\mathcal{A}_t \leftarrow \mathcal{C}(\mathcal{F}_t^-, \phi_t)$ 
5:    $\Delta\Phi_t \leftarrow \text{AGGREGATEDIFFS}(\mathcal{A}_t)$ 
6:    $\tilde{\phi}_t \leftarrow \text{APPLY}(\phi_t, \Delta\Phi_t)$ 
7:   if  $\text{VALIDATE}(\theta_t, \phi_t, \tilde{\phi}_t)$  is non-regressive then
8:     commit  $\Delta\Phi_t$  to the registry and set  $\phi_t \leftarrow \tilde{\phi}_t$ 
9:   else
10:    reject  $\Delta\Phi_t$  and keep  $\phi_t$ 
11:   end if
12:   refresh  $\mathcal{F}_t^-$  under  $(\theta_t, \phi_t)$ 
13: end for
14:  $\mathcal{D}_t \leftarrow \text{COLLECTTRAJECTORIES}(\theta_t, \phi_t)$ 
15:  $\theta_{t+1} \leftarrow \text{SFT}(\theta_t, \mathcal{D}_t)$ 
16: return evolved Harness  $\phi_t^* \equiv \phi_t$ , updated model  $\theta_{t+1}$ 
```

表 4: 哈内斯批评家使用的失败归因分类法。前五类是可操作的哈内斯失败；智能体错误是非哈内斯弃权标签。

根本原因	主要位置	典型症状
提示歧义 P	指令欠明确或目标冲突	工具模式错误 T 无效工具调用、错误参数模式、后端不匹配 技能缺失 S 缺少可复用例程或分解原语 中间件不匹配 Mid 有缺陷的循环协议、钩子行为或上下文管理 内存溢出 M 持久状态溢出、陈旧内存或检索失败 智能体错误 - 哈内斯检查后失败仍归因于模型侧

失败归因分类法。

表 4 定义了全文使用的归因模式。前五行是可操作的哈内斯类别，每行对应五个哈内斯维度之一。当失败最好由模型而非哈内斯解释时，哈内斯批评家也可弃权为智能体错误；此类情况被排除在哈内斯补丁生成之外。

结构化归因输出。对于每条失败轨迹，HarnessCritic 生成一条结构化记录，形式为 { 根因、harness 维度、严重程度、证据、差异建议 }。证据字段将诊断锚定到具体的轨迹事件，而差异建议则指定一个局部补丁目标，例如工具模式字段、中间件钩子插入点或编排器策略。使用结构化输出使搜索空间可审计，并使聚合操作在显式的补丁位置上进行，而非在自由形式的自然语言上进行。

聚合为 Harness 差异。单次失败可能带有噪声，因此 Co-Harness 在编辑 Harness 之前会跨批次聚合归因。我们根据复发频率、严重程度和建议字段路径的一致性对候选编辑进行排序，然后将兼容的建议合并为单个 Harness 差异 $\Delta\Phi_t$ 。差异被有意设计为局部性的：它仅记录发生变化的字段及其旧值和新值。这种局部性对于可解释性和回滚都至关重要。与开放式的代码空间搜索相比，HarnessCritic 执行基于观察到的失败的有针对性的修复。

3.4 模型对齐循环

一旦 Harness 被进化到 ϕ_t^* ，我们在该配置下重新运行当前模型，并收集生成的轨迹集 $\mathcal{D}_t$ 。 $\mathcal{D}_t$ 的每个元素都包含监督所需的完整交互轨迹：提示上下文、工具调用、工具响应、中间决策以及最终验证结果。我们保留满足任务验证器或符合基准相关质量过滤器的轨迹。

模型更新随后在 $\mathcal{D}_t$ 上微调 θ_t 以获得 θ_{t+1} ：

$$\theta_{t+1} = \operatorname{argmax}_{\theta} \sum_{(x, \tau) \in \mathcal{D}_t} \log p_{\theta}(\tau | x).$$

直观地说，这一步将改进后的 Harness 所支持的行为蒸馏到模型参数中。更新后的模型通常能够以更轻的中间件干预、更短的恢复链或更可靠的工具使用来解决任务，从而减少未来的 Harness 债务。更重要的是，更强的模型会暴露出以前被较弱的基础行为所掩盖的新失败模式，为下一轮 Co-Harness 提供更有意义的优化问题。

3.5 补丁验证与 Harness 注册表

提出的 Harness 补丁只有在改善目标失效模式（ $\delta_{\text{in}} > 0$ ）且不导致保留行为（ $\delta_{\text{out}} \geq 0$ ）退化时才会被接受；被接受的补丁会写入带版本的 Harness 注册表，该注册表作为审计追踪并支持完全回滚（验证协议和注册表细节见附录 E）。

3.6 为什么循环会复合

双循环只有在三个条件同时满足时才会复合。第一，Harness 更新必须提高轨迹质量，而不仅仅是改变表面格式。第二，模型更新必须内化这些改进，使下一个模型真正更强，而不是仅仅依赖脆弱的脚手架。第三，更强的模型必须通过使更先进的中间件、技能或记忆机制值得部署来解锁新的 Harness 机会。如果这些条件中任何一个失败，该过程就会退化为收益递减的单循环优化。

这一视角也解释了为什么 Co-Harness 不同于先前的推理时 Harness 搜索。目标不仅是为当前模型找到更好的外部脚手架，而是创建一个反馈循环，其中更好的脚手架产生更好的训练信号，而更好的模型反过来扩大有用的脚手架修订集合。该机制是我们实验中检验的核心假设。

4 实验

4.1 设置

任务设置：工具集成推理（Tool Integrated Reasoning (TIR)）。所有实验都采用 TIR 范式：智能体通过将思维链推理与对 Python 代码解释器的迭代调用交织来解决每个数学竞赛问题。Harness 缺陷——例如错误的工具模式或缺失的重试钩子——直接转化为模型仅靠推理无法恢复的失败轨迹，使 TIR 成为 Harness 协同进化的特别尖锐的测试平台（完整循环结构和失效模式分类见附录 A）。

Benchmarks. We evaluate on three competitive mathematical reasoning benchmarks of varying difficulty: AIME 2024 (30 problems), AIME 2025 (30 problems), and HMMT February 2025 (30 problems from the individual round). These benchmarks span a range from moderately challenging (AIME24) to extremely

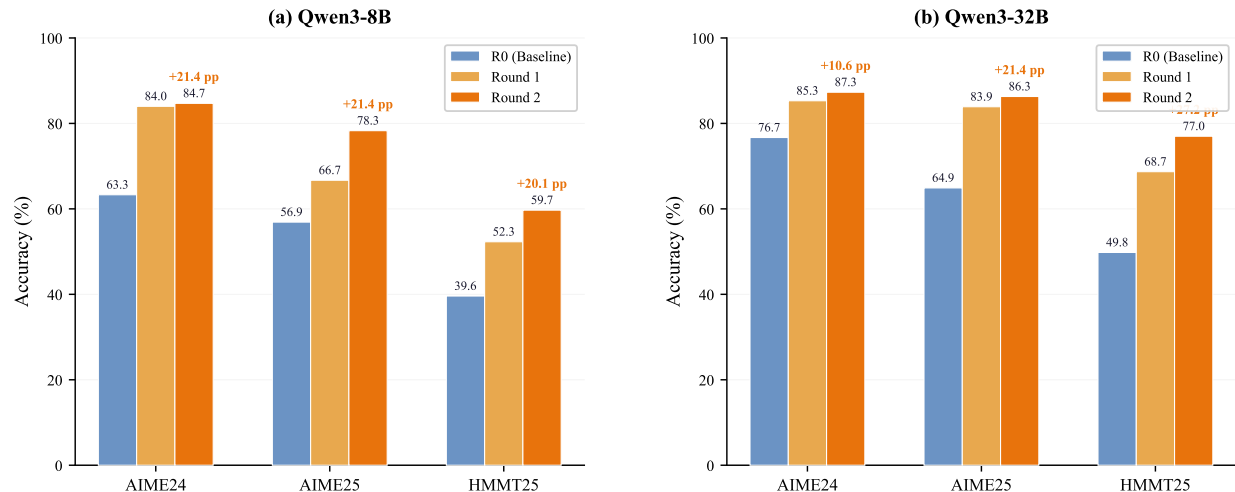


图 5：协同进化双环协同进化结果在不同基准和模型规模上的表现。 分组柱状图展示了 Qwen3-8B（左）和 Qwen3-32B（右）在 AIME24、AIME25 和 HMMT25 上的准确率（%），跨越三个协同进化轮次。每组包含三个柱：基线（第 0 轮，仅进化工具，无监督微调）、第 1 轮（一个完整的协同进化轮次：工具评价器+监督微调）和第 2 轮（两个完整轮次）。第 2 轮柱上方的增量值表示相对于基线的累积改进。协同进化在所有基准和模型规模上均持续提升准确率，且在更难的数据集（HMMT25）上增益更大。

困难（HMMT25），使我们能够评估工具进化收益如何随任务难度变化。所有问题均在上述工具集成推理设置下求解；报告的准确率为 pass@1，对每个问题多次采样取平均。

基础模型。 我们实验了两种模型规模：Qwen3-8B 和 Qwen3-32B。两种模型均具备强大的数学推理先验，并支持工具增强生成，使其成为实际工具集成推理智能体部署的代表性选择。

协同进化轮次。 对于双环实验，我们运行 $T = 2$ 个完整的协同进化轮次（第 1 轮和第 2 轮），每轮包含一次协同进化环传递（工具评价器， $K = 5$ 次内部迭代），随后进行一次模型对齐环传递（对收集的轨迹集进行监督微调）。第 0 轮（基线）仅应用一次工具评价器传递以获得 ϕ_0^* ，但不进行监督微调，作为衡量复合增益的起点。

基线。

- **基线（第 0 轮）：** 由一次 HarnessCritic 演化得到的 Harness (ϕ_0^*)，但未进行 SFT 模型对齐。
- **人工：** 人工设计的静态 Harness，无模型对齐，作为人工工程的上限。
- **协同 Harness（本文方法）：** 如第 3.2 节所述的全双环协同演化。

4.2 双环协同演化：核心实验

图 5 和表 6 展示了核心实验。我们观察到三个关键发现：

表 6：协同 Harness 双环协同演化结果。数学推理基准上的准确率（%）。人工[†]：人工设计的静态 Harness（无模型对齐）。**R0**：基线——由 HarnessCritic 演化的 Harness（ ϕ_0^* ），未进行 SFT 模型对齐。**R1**：经过一轮 HarnessCritic 演化 + SFT 后。**R2**：经过两轮后（本文方法）。 **$\Delta(\text{R0})$** ：R2 相对于基线的累积增益。 **$\Delta(\text{Hu})$** ：R2 相对于人工设计 Harness 的增益。最佳结果以粗体显示。

Model	Benchmark	Human [†]	R0	R1	R2	$\Delta(\text{R0})$	$\Delta(\text{Hu})$
Qwen3-8B	AIME24	59.3	63.3	84.0	84.7	+21.4	+25.4
	AIME25	51.3	56.9	66.7	78.3	+21.4	+27.0
	HMMT25	34.7	39.6	52.3	59.7	+20.1	+25.0
Qwen3-32B	AIME24	72.0	76.7	85.3	87.3	+10.6	+15.3
	AIME25	61.3	64.9	83.9	86.3	+21.4	+25.0
	HMMT25	46.7	49.8	68.7	77.0	+27.2	+30.3
Average		54.2	58.5	73.5	78.9	+20.4	+24.7

持续累积增益并超越人工设计的 Harness。协同进化框架（Co-Harness）在两种模型规模和全部三个基准上均实现了跨轮次的单调准确率提升。平均准确率从 58.5%（基线）提升至 73.5%（第 1 轮，+15.0 个百分点），再提升至 78.9%（第 2 轮，累计+20.4 个百分点），证实了双环协同进化机制能够产生真正的复合式自我改进。关键的是，第 2 轮还以平均+24.7 个百分点的优势超越了人工设计的静态框架（Harness），表明自动化协同进化不仅超越了自身的起点，还突破了精心手工构建的固定配置的上限。

更难的基准受益更多。框架协同进化的收益在 HMMT25（最难的基准）上最为显著：8B 模型提升+20.1 个百分点，32B 模型提升+27.2 个百分点。这证实了核心洞见：当模型面临复杂的多步推理任务时，框架配置的不足更可能导致失败，因此更好的框架配置最为有效。相比之下，AIME24（基线已相对较高）在两个模型上的提升最小。

更大的模型在困难任务上从框架协同进化中受益更多。对于 32B 模型，HMMT25 的提升（+27.2 个百分点）显著超过 AIME24 的提升（+10.6 个百分点）。这表明更强的模型具有更大的潜在能力，可以通过更好的框架释放——这正是第 3.6 节所描述的机制。

4.3 自主框架进化：AIME24 案例研究

图 7 展示了在 AIME24 上使用 Qwen3-8B 的完整、自主的 22 版本框架进化轨迹。从崩溃的初始配置（32 秒内出现 vllm 键值缓存错误）开始，协同进化框架循环自主地经历了三个阶段：

1. 阶段 1——工程修复（初始版本至 v7）：框架批评器将初始崩溃归因于工具模式错误，修补了启动配置，并通过从线程池切换到进程池解决了僵尸线程缺陷。第一次完整评估（v3）在 3.78 小时内达到了 59.6%。
2. 第二阶段——效率优化（v12 – v15）：全局批量推理将墙钟时间缩短了 $8.7\times$ （3.78 h $\rightarrow$ 1.11 小时），同时在后续轮次恢复思维链使准确率保持在 61.5%。
3. 第三阶段——集成验证（v18 – v22）：通过 6 轨迹多数投票结合两个不同种子，在总耗时 2.29 小时下取得了 **63.3%** 的最佳结果——这是在 3.33 小时时间预算内达到的最高准确率。

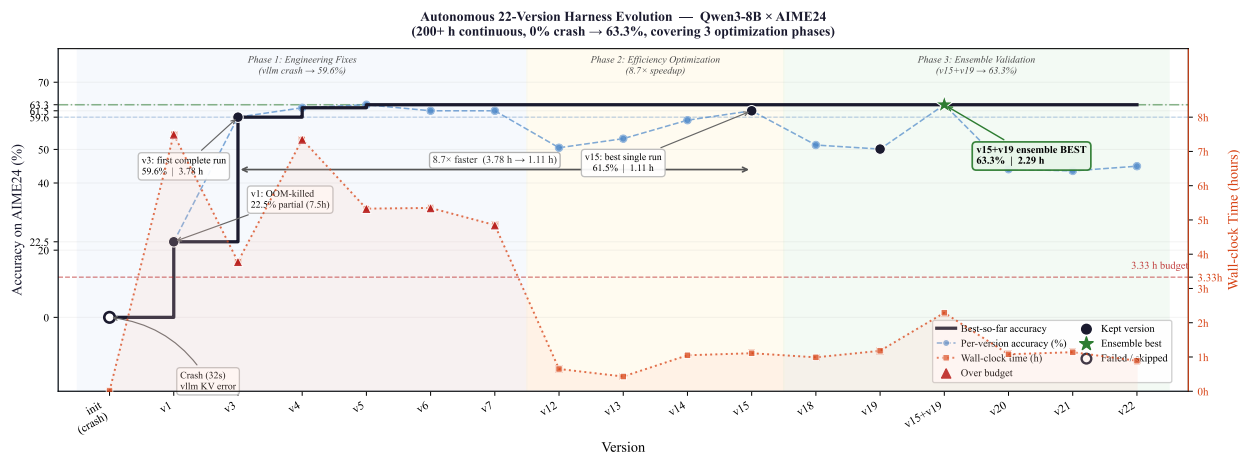


图 7：基于 Qwen3-8B 的 AIME24 上自主 22 版本 Harness 演化。智能体在无人工干预的情况下连续运行超过 200 小时，经历了三个优化阶段：第一阶段（工程修复，0% → 59.6%），第二阶段（效率优化，8.7× 加速），以及第三阶段（集成验证，最终准确率 63.3%）。左轴：准确率（%）；右轴：墙钟时间（小时）。红色虚线：3.33 小时时间预算。绿色星标：最佳集成结果（v15+v19）。

该轨迹表明，HarnessCritic 能够自主权衡准确率与效率，检测并回滚性能退化（v20 – v22：与 Qwen3 内部推理相冲突的领域特定提示），并发现诸如集成策略等非显而易见的改进。

5 分析与讨论

8B 饱和效应。对于 Qwen3-8B 在 AIME24 上的表现，第二轮（84.7%）相比第一轮（84.0%）提升甚微。我们将此归因于两个因素：（1）8B 模型在该基准上的推理能力已接近其上限，限制了 Harness 进一步演化的收益；（2）较高的 R0 基线（63.3%）意味着剩余可归因于 Harness 的失败较少——大多数错误现在源于模型内部而非脚手架相关。相比之下，32B 模型在 AIME24 上持续改进（R1 → R2: +2.0 个百分点），因为其更强的推理能力能更好地利用演化后的 Harness。

协同演化何时无法产生复合增益？经验上观察到三种失效模式：（1）监督微调平台期：若模型架构限制进一步推理提升，监督微调增益饱和，下一轮协同驾驭重新进入相同失效分布。在 AIME24 上 8B 模型观察到该迹象。（2）驾驭过度修补：早期轮次中过度接受差异可能引入超出较弱模型承受能力的复杂度。（3）归因级联：一个差异在修复某一缺陷的同时引入另一缺陷，为下一轮创造更复杂的失效分布；版本控制的驾驭注册表允许回滚至最后验证配置。

驾驭债务分析。基于轨迹训练的一个关键问题是驾驭债务：若轨迹在补偿模型弱点的脚手架配置下收集，训练后的模型可能过度依赖该脚手架，在测试时（无驾驭）表现不佳。结果表明协同驾驭降低驾驭债务：在驾驭批评家演化轨迹上训练的模型每轮测试时达到更高绝对准确率，证明轨迹构建可迁移推理技能而非驾驭依赖行为。值得注意的是，8B 模型的 AIME25 准确率从 56.9%（基线）提升至 78.3%（第二轮）——+21.4 个百分点增益反映真实技能习得而非单纯驾驭记忆。

6 结论

本文提出协同驾驭，一种面向语言智能体的双环协同演化框架。该框架交替两个循环：协同驾驭循环利用驾驭批评家通过语义基础失效归因演化智能体脚手架，模型对齐循环在演化驾驭产生的高质量轨迹上微调模型。两个循环形成正反馈螺旋——更好的驾驭产生更好的轨迹，训练更强的模型，进而解锁此前无法实现的进一步驾驭改进。

在工具集成推理基准（AIME24、AIME25、HMMT25）上，两轮协同驾驭双环协同演化在 Qwen3-8B 和 Qwen3-32B 上平均准确率提升+20.4 个百分点，其中 Qwen3-32B 在 HMMT25（最难基准）上获得最大增益+27.2 个百分点——证实更好的驾驭产生更好的轨迹，训练更强的模型，解锁进一步驾驭改进。

核心洞见——脚手架与模型并非相互独立，而是相互依存的优化轴——为智能体训练开辟了新方向。当前范式要么冻结执行框架（SWEsmith、AgentTuning），要么冻结模型（ADAS、OPRO）；协同执行框架是首个同时演化两者的框架，实现了单独任何一方都无法达成的复合式自我改进。

局限性。协同执行框架需要一个能力较强的评判大语言模型、用于冷启动的最小失败轨迹量，以及多轮监督微调所需的可观算力。对于需要反事实控制流推理的失败模式，归因准确率会下降。结构性执行框架的重新设计仍由人类负责。

未来方向。三个扩展方向尤为有前景。在线协同执行框架将在展开过程中实时进行执行框架评判归因，从而在长程任务中实现运行内执行框架自适应。基于强化学习的执行框架演化将以奖励信号驱动搜索取代大语言模型评判器，可能发现非显而易见的配置。多智能体协同执行框架将把协同演化扩展到具有多个专用智能体的系统，其中智能体中间件层（中间维度）成为关键优化目标。这些扩展共同将推动协同执行框架从周期性批处理过程走向连续的、在线的脚手架健康监测器，与模型共同演化。

参考文献

- [1] Dario Amodei. Machines of loving grace. <https://www.darioamodei.com/essay/machines-of-loving-grace>, 2024. Accessed: 2026-04-27. (page 1)
- [2] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023. (page 1)
- [3] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, 2024. (page 1)
- [4] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055): 609–616, 2025. (page 1)

- [5] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025. (page 1)
- [6] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024. (page 1)
- [7] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025. (page 1)
- [8] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, March 2026. URL <http://arxiv.org/abs/2603.28052>. (pages 1 and 3)
- [9] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026. (pages 2 and 3)
- [10] Jiayi Zhang, Yongfeng Gu, Jianhao Ruan, Maojia Song, Yiran Peng, Zhiguang Han, Jinyu Xiang, Zhitao Wang, Caiyin Yang, Yixi Ouyang, et al. Harnessing agentic evolution. *arXiv preprint arXiv:2605.13821*, 2026. (pages 2 and 3)
- [11] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. (page 3)
- [12] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. ReTool: Reinforcement learning for strategic tool use in LLMs. *arXiv preprint arXiv:2504.11536*, 2025. (page 3)
- [13] Xuefeng Li, Haoyang Zou, and Pengfei Liu. ToRL: Scaling tool-integrated RL. *arXiv preprint arXiv:2503.23383*, 2025. (page 3)
- [14] Zirui Yang et al. Why does tool-integrated reasoning work? a formal perspective. *arXiv preprint arXiv:2508.19201*, 2025. (page 3)
- [15] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024. (page 3)

A TIR 智能体设置：多轮代码解释器循环

本文所有实验均采用工具集成推理范式，智能体通过交替进行思维链推理与迭代式 Python 代码解释器调用来解决数学竞赛问题。我们描述了工具集成推理循环的结构以及执行框架如何在其中起中介作用。

工具集成推理（Tool Integrated Reasoning, TIR）循环结构。每个回合开始时，问题陈述被插入系统提示中。随后模型生成一个响应，该响应可能包含零个或多个如下形式的工具调用块：

```
<tool_call> {"name": "python", "arguments": {"code": "..."}}
</tool_call>
```

中间件（Middleware）拦截每个工具调用，将代码块分派给 Python 解释器子进程，并将解释器的响应（标准输出、标准错误和返回值）作为工具结果追加回对话上下文中：

```
<tool_result> {"stdout": "42\n", "stderr": "", "exit_code": 0}
</tool_result>
```

模型随后继续生成，可选择发出更多工具调用或给出最终答案。此过程最多重复 max turns 轮（初始为 15 轮，在案例研究中第二阶段测试框架演进后增加到 30 轮）。

工具集成推理（TIR）轨迹如何揭示测试框架（Harness）的不足。失败的 TIR 轨迹表现出的失败模式与纯文本失败有本质区别：

- 工具模式失败（T）：工具调用的 JSON 格式错误或使用了不支持的参数名，导致解释器以模式验证错误拒绝调用，而非执行代码。这会在上下文中产生无用的错误标记，并常常使回合停滞。
- 中间件失败（Mid）：重试钩子缺失或配置不当，导致瞬时解释器崩溃（如子进程内存不足）使回合静默终止，而非触发重试。模型未收到任何反馈信号。
- 上下文管理失败（Mid）：在模型发出大量工具调用的长回合中，累积的解释器输出溢出上下文窗口。配置不当的压缩策略丢弃了关键中间结果，导致模型重复计算或丢失部分进展。
- 提示失败（P）：系统提示未明确指示模型将代码作为主要推理工具，导致模型将代码调用推迟到最后一步，错失迭代探索与验证的机会。

HarnessCritic 将每条失败轨迹归因于上述类别之一，并生成针对性补丁。AIME24 案例研究（附录 H）在单次自主进化运行中展示了全部四种失败模式。

工具集成推理（TIR）测试框架与纯文本测试框架。在纯文本智能体中，次优提示会逐渐降低响应质量；即使指令不完善，模型通常仍能产生可用的轨迹。而在 TIR 中，单个配置错误的组件即可完全中止回合：若 Python 解释器拒绝工具调用，模型便无法获得计算结果，被迫猜测或给出错误答案。TIR 失败的这种二元性质——要么代码执行并返回有用反馈，要么不执行——为 HarnessCritic 提供了更清晰的梯度，也解释了为何测试框架协同进化在此机制下能带来尤为显著的收益。

B 归因提示模板

以下提示模板用于 HarnessCritic 中基于大语言模型的失败归因。思维链推理（<think> 块）先于结构化 JSON 输出。

系统：你是一名专家智能体脚手架工程师。给定失败的智能体轨迹和当前 Harness 配置，识别失败的根本原因。

```
Output your reasoning in <think>...</think>, then output
a JSON object matching this schema:
{
  "root_cause": one of [
    "prompt_ambiguity", "tool_schema_error", "skill_missing",
    "middleware_mismatch", "memory_overflow", "agent_error"
  ],
  "harness_dim": one of ["P", "T", "S", "Mid", "M"],
  "severity": one of ["critical", "major", "minor"],
  "evidence": "< 1 sentence citing specific trajectory event >",
  "diff_suggestion": {
    "field_path": "middleware.hooks.post_tool_call[0].type",
    "old_value": null,
    "new_value": "retry_on_rate_limit"
  }
}
```

```
== 当前 HARNESS 配置 ==
{harness_config_yaml}
```

```
== 失败轨迹 ==
{trajectory_text}
```

C Harness 配置详情

所有实验中使用的完整 Harness 初始配置 ϕ_0 。注意工具维度配置为 TIR：主要工具是 Python 代码解释器，中间件负责管理多轮代码执行循环。

D 人工标注协议

两位专家标注者（具有 > 2 年智能体工程经验）使用表 4 中的分类法独立标注了 200 条失败轨迹。分歧由第三位专家解决。人工标注者间一致性为 $\kappa = 0.77$ 。标注者被给予 90 分钟处理全部 200 条轨迹，可访问 Harness 配置和完整轨迹日志。未向标注者展示任何大语言模型归因信息。所有归因实验使用基础模型 θ_0 （第 0 轮）。

E 补丁验证协议与 Harness 注册表

验证协议。提出的 Harness 补丁 $\tilde{\phi}_t$ 针对当前 Harness ϕ_t 在两个留出划分上进行验证：一个留入集 $\mathcal{V}_{\text{in}}$ 包含表现出目标失败模式的示例，以及一个留出集

表 8: 初始 Harness ϕ_0 配置 (TIR 设置)。

Dimension	Setting
P(Prompt)	System prompt: 512-token instruction with mathematical reasoning guidelines and TIR usage instructions (“use Python code to assist your reasoning”); no few-shot retrieval
T(Tool)	Python code interpreter (sandboxed subprocess); JSON tool-call schema specifying <code>name: python</code> , <code>arguments.code: str</code> ; default timeout 30s
S(Skill)	None (empty skill set at Round 0)
Mid(Middleware)	Orchestrator: max turns; terminate on <DONE> or final-answer token. Hooks: retry on ToolCallException (max 3, linear backoff). Context management: sliding-window compression keeping the last 4k tokens; interpreter outputs truncated to 2k tokens
M(Memory)	No long-term memory or external retrieval store at Round 0

$\mathcal{V}_{\text{out}}$ 包含先前成功或正交的行为。我们计算

$$\delta_{\text{in}} = R(\mathcal{V}_{\text{in}}; \theta_t, \tilde{\phi}_t) - R(\mathcal{V}_{\text{in}}; \theta_t, \phi_t),$$

$$\delta_{\text{out}} = R(\mathcal{V}_{\text{out}}; \theta_t, \tilde{\phi}_t) - R(\mathcal{V}_{\text{out}}; \theta_t, \phi_t).$$

仅当 $\delta_{\text{in}} > 0$ 且 $\delta_{\text{out}} \geq 0$ 时, 补丁才被接受。该规则防止系统对孤立失败过拟合或牺牲先前可靠的行为。被拒绝的补丁立即回滚; 注册表审计跟踪记录接受和拒绝的候选补丁, 以确保可复现性。

Harness 注册表。所有跨共同进化轮次的 Harness 配置都存储在一个版本化的 Harness 注册表中, 使用简单的目录结构:

```
harness_registry/
  v0/ harness.yaml          # phi_0 (initial)
  v1/ harness.yaml          # phi_0^* (after Round 1 HarnessCritic)
    diffs/
      001_hook_retry.yaml
      002_tool_timeout.yaml
      003_prompt_scope.yaml
      004_strategy_maxrounds.yaml
  v2/ harness.yaml          # phi_1^* (after Round 2 HarnessCritic)
    diffs/ ...
  v3/ harness.yaml          # phi_2^* (after Round 3 HarnessCritic)
    diffs/ ...
```

每个差异文件都是一个自包含的 YAML 补丁, 包含元数据 (轮次、归因来源、验证增量), 支持完整回滚和实验可复现性。该注册表还可作为跨协同进化轮次变更内容及其原因的可解释审计追踪。

F 扩展分析

为什么工具集成推理 (Tool Integrated Reasoning, TIR) 中特别关注线束协同进化? TIR 范式使 Harness 成为结构性瓶颈: Harness 介导代码解释器循环的每一步, 从工具调用的方式

序列化并分派到执行结果如何路由回模型上下文以及错误如何触发重试。纯文本推理智能体在提示不理想时可以退回到言语计算；而工具集成推理智能体若遇到无效的工具模式或缺失的重试钩子，则直接失败——解释器拒绝调用，不返回任何结果，回合终止且没有可供模型学习的信号。HarnessCritic 的归因引导修复正是针对这些组件——工具模式、中间件钩子、回合限制——它们的缺陷导致工具集成推理轨迹无论模型能力如何都会失败，这解释了为何即使微小的工具接口修复也能解锁整类此前无法达到的多轮策略。

哪些基准从 Harness 演进中获益最多？ 准确率提升幅度最大的是最难的基准测试（HMMT25：8B 模型提升 20.1 个百分点，32B 模型提升 27.2 个百分点）。复杂的多步工具集成推理问题暴露出更多 Harness 缺陷——长代码解释器会话的重试策略不足、多个执行结果累积时的上下文溢出，以及中间件路由中的策略不匹配——而且更难的问题每个回合还需要更多代码解释器调用，放大了任何单一 Harness 缺陷的代价。较简单的基准测试（AIME24）提升幅度较小但仍有意义，因为可归因于 Harness 的失败模式较少，模型通常可以用更少的代码调用来弥补。

模型规模与协同演化潜力。 32B 模型在困难基准上展现出比 8B 模型更大的绝对增益（HMMT25：+27.2 个百分点，对比+20.1 个百分点），这表明更强的模型拥有更大的潜在能力，可以通过更好的 Harness 来释放——即第 3.6 节所述的复合机制。两个模型在 AIME25 上取得了相近的增益（各+21.4 个百分点），这表明 Harness 与模型的交互作用取决于每个基准-模型组合的具体失败分布。

G 协同演化轨迹分析

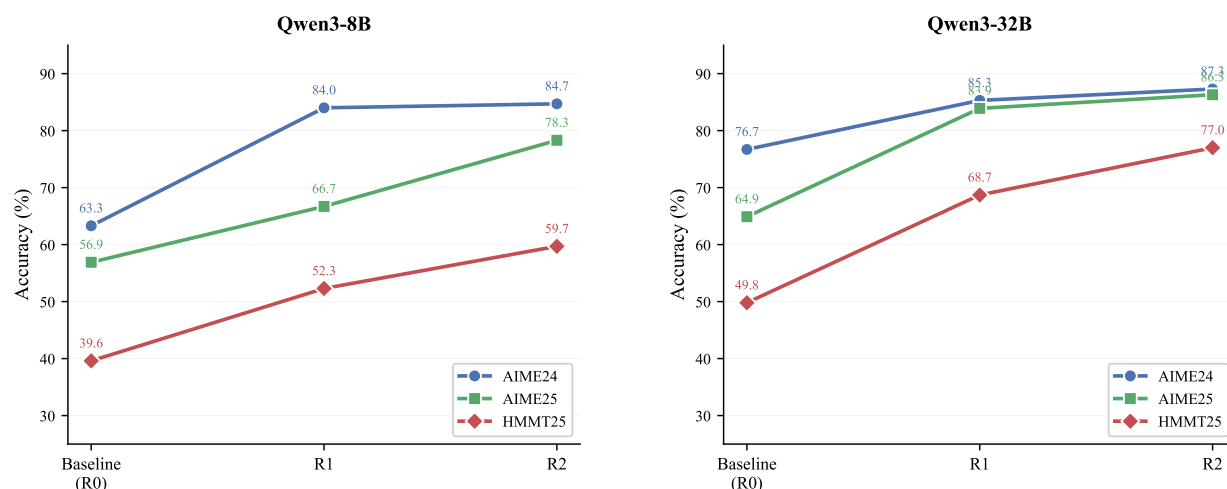


图 9：准确率与协同演化轮次的关系。 折线图展示了各基准在不同轮次上的准确率进展。左侧为 Qwen3-8B，右侧为 Qwen3-32B。所有曲线均呈现一致的上升趋势，最陡峭的增益通常出现在基线与第 1 轮之间。

图 9 可视化了各基准在协同演化轮次中的准确率轨迹。出现了两种模式：

增益递减但持续存在。 最大的准确率跃升发生在基线与第 1 轮之间（平均+15.0 个百分点），从第 1 轮到第 2 轮的增益较小但仍有意义（+5.4 个百分点）。这与复合条件分析一致：最容易的 Harness 缺陷首先被修复，后续轮次则处理越来越细微的失败模式。

8B 模型在 AIME24 上趋于饱和。对于 Qwen3-8B 在 AIME24 上的表现，第 2 轮（84.7%）相比第 1 轮（84.0%）仅有微小提升，这表明 8B 模型在该基准上已接近其推理能力上限。相比之下，32B 模型继续提升（在 AIME24 上从第 1 轮到第 2 轮+2.0 个百分点），这表明更强的模型能更好地利用演化后的 Harness。

H 案例研究：在 AIME24 上的 22 版本自主演化

为说明协同循环在单次长时间运行中的行为，我们追踪了在 AIME 2024 数学推理基准上使用 Qwen3-8B 的完整演化轨迹。该智能体自主运行超过 200 小时，在无人工干预的情况下产生了 22 个 Harness 版本。完整轨迹如图 7 所示；本文在此提供逐阶段的详细分析。

阶段 1：工程修复（版本 init - v7）。初始配置（init）在 32 秒内崩溃，原因是 vllm 后端中 gpu util 参数不匹配，导致 KV 缓存被分配到低于最小阈值。HarnessCritic 将此归因于工具模式错误，并修补了启动配置。下一次运行（v1）暴露了一个僵尸线程缺陷，该缺陷在 7.5 小时后于第 55 题导致内存溢出（部分得分：22.5%）。在从线程池切换到进程池、采用基于 SIGKILL 的清理并启用思维链（thinking=ON），之后，v3 在 3.78 小时内完成了全部 90 道题，准确率为 59.6%——这是首次有效评估。后续变体（v4 - v7）探索了更高的采样数（ $n=5$ ）、系统提示变体和温度调度，最高达到 63.3% 的准确率（v5），但均超出了 3.33 小时的时间预算。

阶段 2：效率优化（版本 v12 - v15）。HarnessCritic 将逐题顺序执行确定为主要延迟瓶颈。切换到全局批量推理（v12 - v14）将墙钟时间从 3.78 小时降至 1.1 小时以下——实现了 8.7× 倍的加速——但代价是准确率暂时下降（v12：50.5%，归因于在后续轮次中无意禁用了思维链）。全程恢复 thinking=ON 并将最大轮数从 15 扩展到 30 后，准确率得以恢复（v15：61.5%，1.11 小时），现已轻松满足预算要求。

阶段 3：集成验证（版本 v18 - v22）。在建立了快速且符合预算的单次运行之后，HarnessCritic 将重点转向轨迹多样性。运行一个独立种子（v19：seed=42, 1.18 小时，单独准确率 50.1%）并将其与 v15 通过 6 轨迹多数投票相结合，将集成得分提高到 **63.3%**，总耗时 2.29 小时——这是在预算内达到的最高准确率。进一步尝试添加领域特定系统提示（v20 - v22）使准确率下降了 ~16.5 个百分点，因为注入的指令与 Qwen3 的内部推理格式相冲突；HarnessCritic 正确地将此识别为提示歧义回归并进行了回滚。

要点。该轨迹在自然场景下展示了协同约束循环（Co-Harness Loop）的三个特性：（1）正确性优先修复——该循环在优化性能指标之前，可靠地修复结构性崩溃和部分失效；（2）帕累托感知搜索——约束批评器（HarnessCritic）将墙钟时间视为显式约束而非单纯度量，从而在准确率与效率之间进行权衡；（3）回滚安全——过度修补（v20 - v22）导致的性能退化由验证重放检测并回退，防止最终配置质量下降。